



GPU optimizations to accelerate an image enhancement algorithm

Amin Daemdoost¹ · Asadollah Shahbahrani^{1,2} · Hesam Noorpoor¹ ·
Nima Esmi^{2,3} · Reza Hassanpour^{2,3}

Accepted: 12 May 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

The quality of images captured in challenging environmental conditions, such as nighttime or low illumination, is often compromised. Low-light images captured in non-uniform illumination environments contain a compressed dynamic range that can be improved to expose more detail and information. Many enhancement algorithms exist for low-light or dark images, but some are computationally intensive, making them difficult to use in real-time scenarios. Unsharp masking is a popular algorithm that enhances details and edges in images. For example, an algorithm incorporates local and global enhancements, including color space conversions from RGB to HSV and back, unsharp masking, and histogram equalization. However, runtime profiling revealed that unsharp masking is computationally expensive, taking up to 75% of the runtime. To address this issue, the unsharp masking steps were implemented on GPU hardware using a straightforward method, with each thread computing a single output to leverage its parallelization power. Subsequently, we analyzed the runtime and identified optimal optimization combinations that leverage shared memory, registers, and changing thread workloads to enhance communication efficiency within the GPU. For a filter size 5×5 , our experimental results showed a speedup of up to 16.36× at the kernel level and 3.5× at the application level using GPU optimization techniques compared to the CPU platform.

Keywords Low-light and dark image enhancement algorithms · GPU · CUDA · Convolution and blurring functions

1 Introduction

In digital image processing, enhancing the quality of images remains a significant challenge. Images often suffer from unwanted artifacts or low contrast, affecting visibility and the ability to extract features in computer vision tasks. Various

Extended author information available on the last page of the article

Published online: 10 June 2025

Springer

techniques, such as contrast stretching, histogram equalization (HE), and gamma correction, have improved image quality [1]. Other methods, such as CNNs and transformers, have been effective in more challenging environments, such as underwater, by enhancing image quality through iterative learning of global features [2]. Among these, unsharp masking is a widely used technique that plays a crucial role in enhancing the sharpness and detail of images, often complementing other methods. This helps enhance images captured under demanding conditions, such as low light, noise, or adverse weather while preserving better information in the output. This is vital for applications in surveillance systems, the automotive industry, and intelligent transportation systems [3–6].

Unsharp masking can be employed in various stages of image processing methods. It can serve as a preprocessing step to highlight edges and high-frequency details [7, 8], or as a post-processing step to address blurred edges or lost details, particularly in challenging conditions like underwater or low-light environments [9–11]. Additionally, unsharp masking is often used as an intermediate step between other enhancement methods [12] or within convolutional neural networks to aid in feature extraction [13, 14].

One of the challenging criteria in image processing is to increase visibility and extract information from low-light images. In conventional methods, a simple HE is often used to improve the contrast of the image, helping with highlighting hidden information in dark images [1]. However, this usually leads to blurred edges and degraded structures. Methods similar to unsharp masking can mitigate this issue by enhancing edge sharpness in the image. However, the approach to combining this with HE is crucial to prevent quality degradation. HE is a global enhancement method that aims to normalize the probability distribution of pixel values. By first applying a local enhancement technique like unsharp masking to emphasize significant details in the image, the entire image can then be normalized using HE or similar global enhancement methods more effectively [15]. As shown in Fig. 1, this algorithm enhances images from various datasets across different applications. Specifically, Fig. 1a depicts a nighttime cityscape sourced from [16], Fig. 1c presents a dash-cam view of a street from [17], Fig. 1e captures a satellite image from [18], and Fig. 1g represents a license plate dataset from [19]. However, it becomes more computationally intensive due to including unsharp masking, which contains a convolution function. For example, Fig. 2 illustrates using the algorithm for real-time dash-cam image enhancement. Dark image enhancement is often used as a preprocessing method, and the main stage, like a detection model, comes after it, so meeting the maximum latency requirement for the enhancement should be set more strictly. Employing the OpenCV function enabled video processing in a resolution of 1080×1920 at 32 frames per second, which makes it hard to meet the needs of a real-time application as the preprocessing step, motivating the acceleration of the computationally intensive steps of the algorithm in alternative hardware like GPU.

A runtime profile was conducted to enhance the performance of the dark image local–global enhancement algorithm [15]¹. The analysis indicated that unsharp

¹ www.youtube.com/cardashcameras.

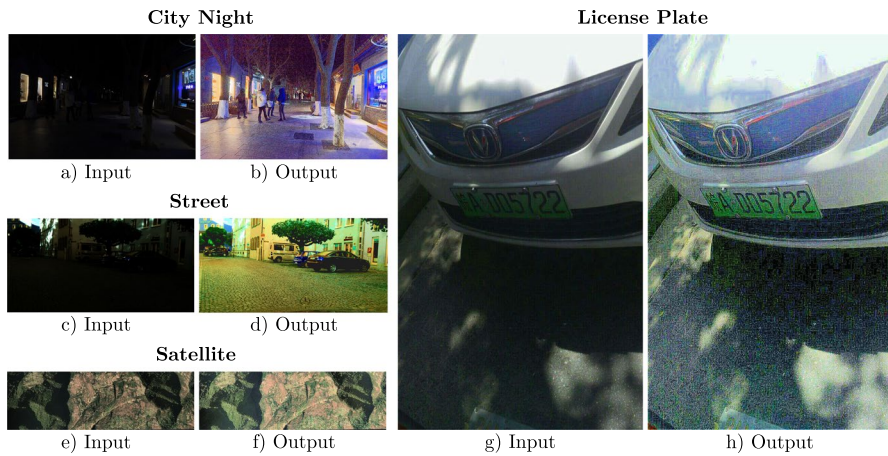


Fig. 1 Enhancing sample images from various scenarios. **(a)** presents a sample image from the dataset referenced in [16], enhanced to produce **(b)**. Similarly, **(c)** is sourced from [17] and has been enhanced to generate **(d)**. **(e)**, taken from [18], is enhanced to create **(f)**. Finally, **(g)** is a license plate dataset from [19] and has been enhanced to produce **(h)**

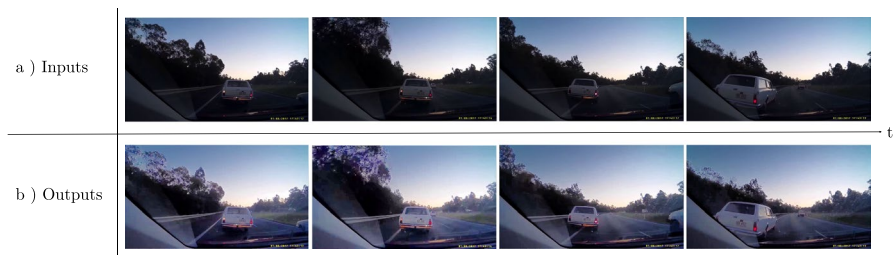


Fig. 2 Sample frames labeled as **(a)** from a dash-cam video, in a resolution of 1080×1920 , undergo enhancement using the local-global algorithm to produce the improved output frames labeled as **(b)**

masking constituted the most significant portion of the runtime up to 75%, with the convolution alone contributing to approximately 39% of the total processing time. Multiple trials of the convolution kernel were conducted on the GPU, utilizing the CUDA parallel programming model. Initially, a straightforward implementation was tested in which each thread computes one output pixel. It achieved a speedup of approximately $9\times$ compared to the CPU. However, further improvement is possible by enhancing data communication efficiency within the GPU. To address this, we tested various optimization methods and analyzed the results to identify the most promising combinations for this task. (i) prefetching data in shared memory (SM), (ii) using registers for data prefetching and increasing each thread's workload by processing multiple output pixels per thread (MOPT), and (iii) combining techniques of SM and MOPT. The experimental results demonstrate that for a 5×5 filter, the highly optimized approach achieves a speedup of approximately $3\times$ over straightforward implementations, up to $7.2\times$ over the CUDA deep neural network

library (CuDNN), and up to $16.36\times$ over CPU implementation using OpenCV functions. In the final evaluation, other steps involved in performing unsharp masking were implemented on the GPU to test the application-level speedup, reaching up to $3.5\times$ over the CPU.

The paper is structured as follows. Section 2 provides some background information and previous studies related to our work. Section 3 shows runtime profiling data on the enhancement algorithm on the CPU implemented using OpenCV functions. Section 4 presents GPU implementations of convolution and optimization techniques. Section 5 presents our experimental setup and details on the results. Section 6 concludes the article and outlines potential directions for future work.

2 Backgrounds and related works

This section first demonstrates the dark image enhancement algorithm and then discusses the optimization methods used in previous studies to accelerate parts of the unsharp masking process on the GPU.

2.1 Dark image local–global enhancement

The algorithm [15] consists of multiple steps, each associated with a time complexity in a sequential implementation, as detailed below.

1. Convert the image from RGB to HSV color space ($O(W \times H)$).
2. Enhance the image locally using an unsharp masking
 - (a) Blur the V channel using a Gaussian filter as defined in Eq. (2) ($O(W \times H \times \text{filter_size}^2)$).
 - (b) Subtract the blurred V channel from the original V channel to obtain the mask given in Eq. (3) ($O(W \times H)$).
 - (c) Sharpen the V channel by adding the weighted mask to it as demonstrated in Eq. (4) ($O(W \times H)$).
3. Globally enhance the image quality by applying a histogram equalization method ($O(W \times H)$).
4. Convert the final HSV image to RGB to obtain the output ($O(W \times H)$).

Initially, the image is converted into the HSV color space to better differentiate brightness (V channel) from other aspects of the image. Following this, a local enhancement is applied to the V channel, which functions as unsharp masking if the weight is set to 1. This requires a Gaussian filter, with coefficients defined in Eq. (1). The standard deviation σ denotes the curve of the Gaussian function. It is usually set to 1 but can be adjusted to increase or decrease the smoothing effect. The indices k and l of the Gaussian filter range from $-\frac{\text{filter size}}{2}$ to $+\lfloor \frac{\text{filter size}}{2} \rfloor$. Then, global

enhancement is applied to the locally enhanced V channel, adjusting pixel intensities for better balance and visibility, as all the steps demonstrated in Fig. 3.

$$G(k, l) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{k^2 + l^2}{2\sigma^2}\right) \quad (1)$$

$$f_c = \frac{\text{filter size}}{2} \text{blur}(i, j) = \sum_{k=-f_c}^{f_c} \sum_{l=-f_c}^{f_c} V(i+k, j+l) \cdot G(k, l) \quad (2)$$

$$\text{mask}(i, j) = V(i, j) - \text{blur}(i, j) \quad (3)$$

$$\text{sharpened}(i, j) = V(i, j) + \text{weight} \cdot \text{mask}(i, j) \quad (4)$$

An image from the ImageNet dataset, shown in Fig. 4a, is darkened in Fig. 4b to demonstrate the effect of the algorithm. Figure 4c, d, and e illustrate the output with different weights. The results indicate a significant increase in the mean value of the darkened image, along with enhanced edge details in the output images. The sharpness level is adjusted based on the applied weight. The structural similarity index measure (SSIM) and peak signal-to-noise ratio (PSNR) show how similar the enhancement results are to the original bright image.

2.2 CUDA platform

Compute unified device architecture (CUDA), presented by NVIDIA, uses an application programming interface approach to enable general-purpose processing and parallel computing on NVIDIA GPUs. The GPU and the CPU can be

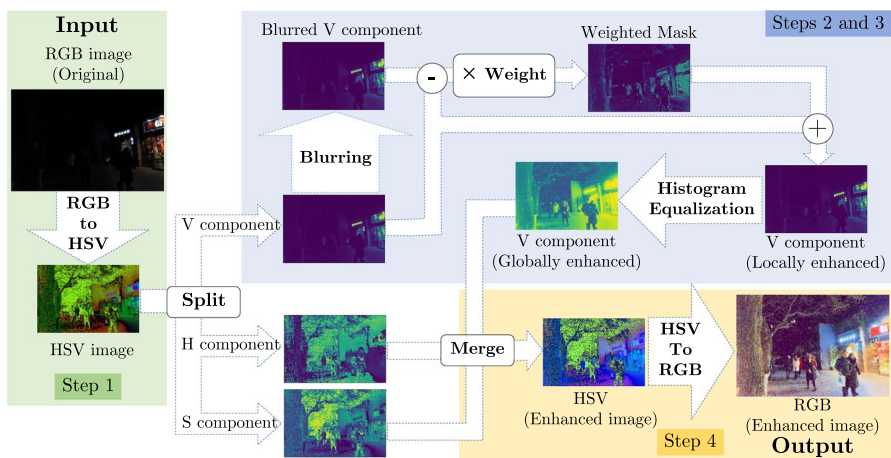


Fig. 3 Process of the dark image local–global enhancement algorithm

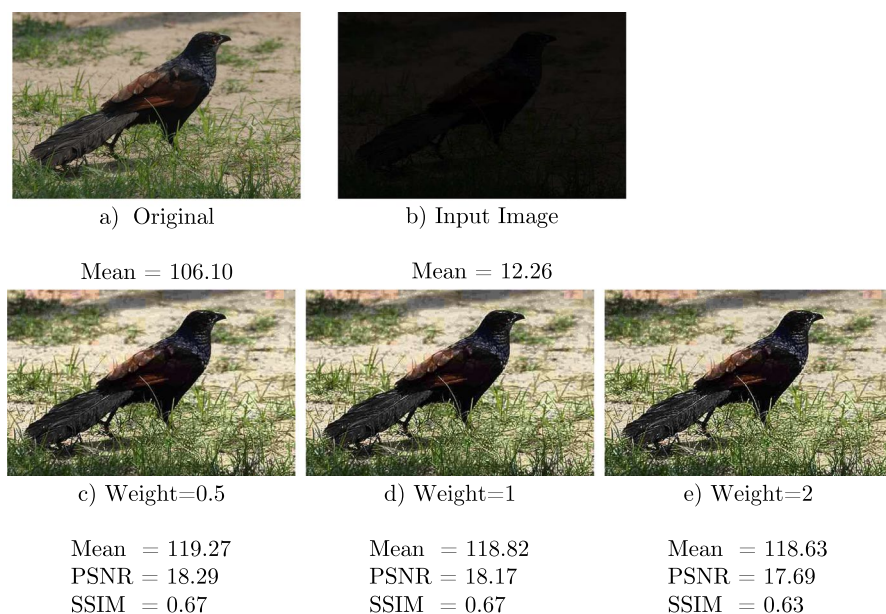
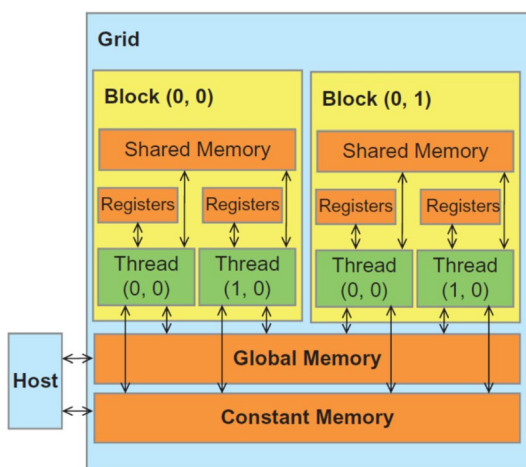


Fig. 4 Evaluation of a sample image from the ImageNet dataset after synthetic darkening. The mean value of each image, along with PSNR and SSIM metrics between the original and enhanced images, is reported. **a** Original image, **b** darkened image used as input to the algorithm, **c** enhanced output with weight = 0.5, **d** enhanced output with weight = 1, **e** enhanced output with weight = 2

Fig. 5 Memory hierarchy within a GPU includes different levels, such as global memory, shared memory, and registers [20]



used through CUDA, which offers two sets of low-level and high-level APIs to facilitate communication. The overhead time required for data transfers from the CPU to the GPU and back again is a drawback of this method. A GPU has several levels of memory, as demonstrated in Fig. 5. Only data stored in the constant

and global memory can be written to or read by a CPU. Global memory is the largest area of a GPU grid, but it suffers from high latency. To mitigate this problem, other memory levels, which are smaller but faster, were placed close to threads in each block as shared memory and registers. However, because of their limited capacity, CUDA systems face resource management challenges since ineffective resource management could prevent threads from operating simultaneously. Another level of this model involves constant memory, a read-only memory that allows aggressive caching in the L1 caches because it does not present cache coherence issues during execution.

2.3 Related works

In GPU implementation, many parameters affect the kernel's efficiency, such as the number of registers, shared memory, and block size. Therefore, optimizing methods suitable for the task is necessary to utilize the resources more effectively and achieve higher speedups. These techniques can be categorized into the following four criteria [21].

- **Memory Access:** GPUs can offer high computational power due to a high degree of parallelism, which makes interacting with slow memory a challenging issue that can limit performance.
 - **On-chip memory techniques** Concentrates on the memories on the GPU chip itself. These techniques aim to make better use of the memory hierarchy by taking advantage of memories such as constant memory, shared memory, and registers.
 - **Off-chip memory techniques** Relates more to device memory, which is typically much slower than on-chip memory. They aim to improve access to device memory located off the GPU chip.
- **Irregularity:** GPUs are highly regular architectures as their control units are simplified to give room for more computation units, so another important criterion is how to efficiently map irregular situations like branches to the hardware. This is especially challenging for irregular memory access, which is very expensive on GPUs.
- **Balancing:** GPUs are designed with simple control units. So, mapping parallelism algorithms to hardware as efficiently as possible to balance resource usage is crucial.
 - **Balancing the Instruction Stream:** Deals with techniques that aim to balance the usage of resources on the GPU by using fused or fast math functions, decreasing branch divergence, etc.
 - **Parallelism-related Balancing:** It also covers techniques for load balancing, which is important for ensuring that all cores are used efficiently.
 - **Synchronization-related Balancing:** Synchronization is essential to GPU programming. However, it can also be a performance bottleneck, making techniques that minimize synchronization crucial.

- **Host Interaction:** Includes the techniques to optimize the communication between the CPU and GPU using methods like unified memory view of the CPU and GPU memory, pinned memory, and pinned and/or mapped memory to overlap communication with computation.

Standard unsharp masking involves a computationally intensive convolution function followed by matrix operations to create the mask and sharpened output. Researchers have used GPUs to improve convolution performance, as shown in studies like [22–26]. One way to use GPUs is through libraries like CuDNN that offer predefined neural network functions [27]. However, better performance can be achieved with optimization methods customized for the task.

One of the most challenging problems in convolution implementation is excessive global memory access. Due to filter coefficients being constant, studies like [28–33] used constant memory to store the filter. To reduce repeated computations, [31, 33] used the registers to save intermediate data that can be reused. Two subsequent studies utilized row and column reusing algorithms like recursive filters, reducing computational complexity and taking advantage of data overlap between operations [34, 35]. Another optimization technique in memory access criteria is prefetching reusable data in other memory hierarchies. A prefetching can be performed using registers to load parts of data into each thread's registers for row and column reusing [36, 37]. Another study used registers to load sufficient input data for calculating multiple outputs, requiring fewer threads and increasing thread works, therefore minimizing communications between global memory [29]. Using shared memory to prefetch data was another way to reduce global memory access, as demonstrated in [20, 30]. Also, to decrease branch divergence, a technique to modify the kernel and employ loop unrolling techniques based on the filter size was used in [30]. Another approach to improving efficiency involves the use of approximation methods. Articles such as [38] employ techniques like Winograd convolution to exploit the massive parallel processing capabilities of GPUs, facilitating convolution calculations through a sequence of element-wise multiplications. However, due to numerical stability issues, this method is commonly used for small filters, such as 3×3 .

Table 1 summarizes various optimization techniques and their achieved speedups. The table demonstrates a recurring trend of using memory access optimization techniques, highlighting the potential of utilizing the memory hierarchy more effectively.

3 Profiling dark image local–global enhancement

To identify performance bottlenecks during its execution and optimize code efficiency, we implemented the local–global enhancement algorithm on the CPU using OpenCV functions. Table 2 shows runtime results tested on image sizes between 720×1080 and 1768×5672 . The most time-consuming part of the algorithm was discovered to be the convolution function in the local enhancement, taking up

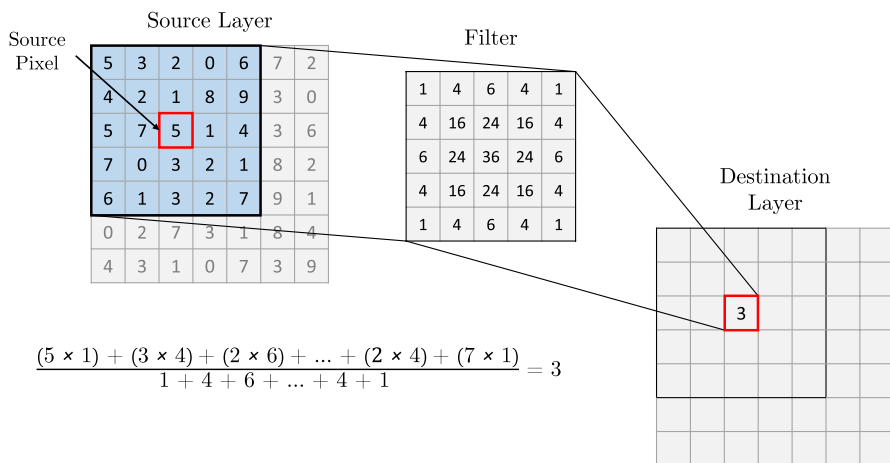
Table 1 An overview of a convolution kernel implemented on a GPU, comparing optimization techniques

Year	Reference	GPU hardware	Optimization category	Optimization techniques	Speedup
2012	[28]	GTX465(x2)	Memory access	Constant memory	–
2013	[29]		Memory access	Constant memory, Local memory, MOPT	up to 8.8× compared to global memory only
2014	[30]	Nvidia GTX680	Memory access, irregularity, balance	Adaptive tiling and loop unrolling, SM, Constant memory	Up to 18.96×
2016	[31]	NVIDIA K40	Memory access	Local memory	3.6× compared to CuDNN
2020	[32]	NVIDIA GTX 1060	Memory access	Constant memory	Up to 15.46× over CPU
2019, 2021	[34, 35]	NVIDIA GeForce GTX TITAN X	Memory access	Algorithm, local memory	–
2020	[36]	NVIDIA 2080Ti	Memory access	Column and row reuse, local memory	Up to 2× speedups over CuDNN
2021	[37]	NVIDIA 2080Ti GPU	Memory access	Column and row reuse, local memory	Up to 3× over CuDNN
2021	[33]	AMD Radeon RX5700, NVIDIA GTX1080	Memory access, balancing	Constant memory, local memory, efficient block size, pinned host memory	up to 18.52× over CPU OpenMP
2023	[38]	AMD MI210	Memory access, balancing	Algorithm, local memory	–
2024	[25]	NVIDIA 920MX	Memory access	Local memory	speedup of 14.54× over CPU

Table 2 A runtime profile of the dark image local–global enhancement algorithm was performed using sample images from the datasets of [16–19]

Image Name	City night [16]	License plate [19]	Street [17]	Satellite [18]
Image size (Pixels)	720 × 1080	1160 × 720	1024 × 2048	1768 × 5672
Total	7.2 (ms)	9.12 (ms)	23.03 (ms)	99.06 (ms)
RGB to HSV	7.2%	9.3%	4.9%	4.6%
Convolution	39.1%	37.3%	37.4%	41.3%
Masking	8.5%	8.3%	15.4%	16.2%
Sharpening	10.9%	9.6%	16.8%	18%
Global enhance	8.1%	10.7%	5.6%	3.6%
HSV to RGB	7.9%	12.6%	6.9%	5.2%
Others (split, merge)	18.3%	12.3%	13%	11.1%

The proportion of the function’s runtime relative to the total execution time was reported

**Fig. 6** An example of applying a 5×5 filter on an image

around 41.3% of the runtime which has been highlighted in the table. Subsequently, the masking and sharpening components, part of the local enhancement, could be fused into a custom function. This motivates improving the performance of all local enhancement steps using alternative accelerators, such as GPUs.

Assuming that the input image has a size of $W \times H$, applying a $S \times S$ filter creates a high computational complexity of $O(W \times H \times S \times S)$. As an example presented in Fig. 6, on a filter size of 5×5 , there are $W \times H \times 25$ multiplications, $W \times H \times 24$ additions, and $W \times H$ divisions, which correspond to $W \times H \times 50$ floating-point operations. Although optimized on the CPU using libraries like OpenCV, it has not reached its maximum parallelization capability. The algorithm consists of independent tasks that can be run fully in parallel, reducing the complexity of each thread to $O(S \times S)$.

4 GPU implementation of convolution

In this section, a straightforward method is initially implemented, followed by a runtime analysis to pinpoint critical performance bottlenecks. Subsequently, three methods are presented, each featuring common optimizations, such as reusing intermediate variables in registers and leveraging constant memory to store the convolution kernel. Additionally, more advanced techniques are incorporated into the methods. The first two methods are based on the most effective existing optimizations for convolution operations, while the third represents the proposed approach, which combines the best features of the previous techniques.

4.1 Straightforward implementation

The convolution kernel was implemented on the GPU using a straightforward method to maximize parallelization by assigning each thread to process one output pixel, as shown in Fig. 7. Additionally, a local variable in each thread was used to calculate the corresponding output pixel in its local registers and then store it in the global memory, as demonstrated in Algorithm 1. The algorithm is designed to handle border pixels by treating pixels outside the border as zeros, effectively implementing a zero-padding method during calculations.

Algorithm 1 Straightforward implementation of convolution kernel on GPU

```

Function(img, filter, blur, W, H, filter_size)
  row  $\leftarrow$  blockIdx.y * blockDim.y + threadIdx.y
  col  $\leftarrow$  blockIdx.x * blockDim.x + threadIdx.x
  f_c  $\leftarrow$   $\lfloor \frac{\text{filter\_size}}{2} \rfloor$ 
  if  $0 \leq \text{row} < H$  and  $0 \leq \text{col} < W$  then
    tmp  $\leftarrow$  0.0
    for k  $\leftarrow$  -f_c to f_c do
      for l  $\leftarrow$  -f_c to f_c do
        if  $0 \leq (\text{row} + k) < H$  and  $0 \leq (\text{col} + l) < W$  then
          tmp  $\leftarrow$  tmp + img[(row + k), (col + l)] * filter[(k + 2), (l + 2)]
        end if
      end for
    end for
    blur[row, col]  $\leftarrow$  tmp
  end if
EndFunction

```

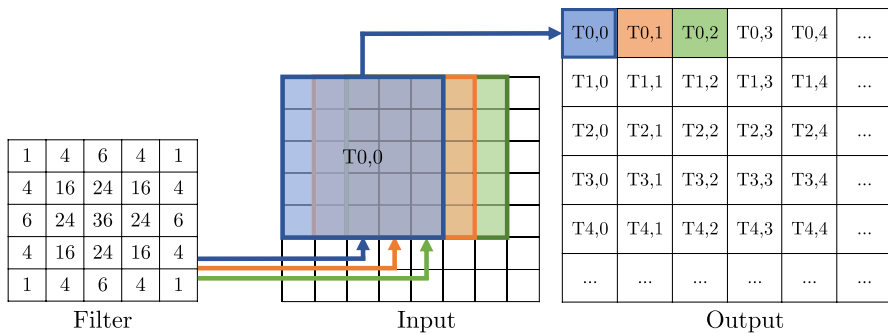


Fig. 7 Assigning tasks to each thread in the straightforward implementation, each computing a single output pixel

4.2 Observations

To evaluate the efficiency of the straightforward implementation, profiling was conducted using NVIDIA Nsight compute [39] on a sample 1024×1024 image processed by a GPU NVIDIA Tesla T4. The results showed a memory throughput of 39.83%, DRAM throughput of 2.79%, and compute throughput of 80.79%. According to the profiler, these parameters represent the percentage of throughput relative to the theoretical ideal throughput, indicating which criteria may limit performance.

The kernel utilizes over 80% of the device's computational capacity, emphasizing the need for optimization to balance workloads and improve efficiency. A more detailed analysis of the compute workload revealed that the arithmetic logic unit (ALU) is the highest-utilized pipeline, operating at 82.4%. Nsight compute analysis suggests the ALU pipeline is overutilized and could be a bottleneck. One way to mitigate this issue is by increasing the floating-point operations executed by each thread, which would reduce the number of threads and lead to a better balance between integer, logic, and floating-point operations.

The sample profiling results indicate low memory and DRAM throughputs, indicating inefficiency of memory accesses. One reason for this is related to issues associated with accessing global memory, including limited bandwidth, high latency, and simultaneous access to the same addresses. Other memory levels, such as caches, registers, constant memory, and shared memory, can be exploited to enhance memory utilization.

4.3 Optimization techniques

4.3.1 Shared memory

The SM optimization uses shared memory in each block to prefetch the required data to calculate the corresponding output pixel. This helps the GPU to reduce data transactions between global memory and threads by bringing the data closer to the computing units, as many pixels need to be accessed multiple times, and the cache is not big enough to store them all. Also, the convolution filter coefficients are stored in constant memory, allowing the GPU to cache the filter more aggressively since it does not need to worry about data variation.

To implement this technique, all data needed by threads in a block must be loaded into shared memory. For example, applying a 5×5 filter in a 12×12 tile requires a block size of 16×16 , with additional threads needed to load all the data for the edge pixels. Then, the threads within the tile range compute the output pixels. The correlation between `tile_size` and `block_size` can be seen in Eq. (5). Figure 8 depicts two subsequent blocks with a size of $M \times M$ and tiles with a size of $N \times N$, illustrating the overlap of data required for their calculations.

$$\text{tile_size} = \text{block_size} - \text{filter_size} + 1 \quad (5)$$

4.3.2 Register-only multiple output per thread

As the input image size increases, more threads are needed, and more global accesses and block switches inside the multiprocessors lower the performance. $R + \text{MOPT}$ technique increases the work done in each thread by calculating several output pixels, as demonstrated in Fig. 9. Additionally, since the data are reused to calculate several outputs, it is prefetched into local variables to take advantage of fast register accesses.

4.3.3 Shared memory multiple output per thread

The proposed SM + MOPT technique combines the best attributes of prior optimizations. It increases each thread's work, but instead of using local variables to prefetch data, it uses shared memory, which decreases the overhead of data loads from global memory by dividing it among multiple threads, thereby reducing the need to load the same data into threads separately and reusing the data between threads. For instance, if each thread processes four output pixels in a block size of 32×32 , the tile size would be 28 (based on Eq. 5). According to Algorithm 2, a shared memory with a 64×64 capacity is required in this setup. Initially, each thread in a block needs to load multiple pixels into shared memory. After synchronization between threads, only threads within the range of the tile size start to calculate the outputs. For further optimization,

- Prefetching data in shared memory.
- Increasing the number of tasks for each thread

Algorithm 2 Implementation of SM + MOPT when computing four outputs per thread

```

1: Function(img, blur, filter, W, H, filter_size, tile_size)
2:  $f\_c \leftarrow \lfloor \frac{\text{filter\_size}}{2} \rfloor$ 
3: Declare shared unsigned char array tileNs[64][64] in a  $32 \times 32$  block.
4:  $tx \leftarrow \text{threadIdx}.x$ 
5:  $ty \leftarrow \text{threadIdx}.y$ 
6:  $row\_o \leftarrow (ty + \text{blockIdx}.y * \text{tile\_size}) * 2$ 
7:  $col\_o \leftarrow (tx + \text{blockIdx}.x * \text{tile\_size}) * 2$ 
8:  $row\_i \leftarrow (row\_o - f\_c)$ 
9:  $col\_i \leftarrow (col\_o - f\_c)$ 
10: for  $i \leftarrow 0$  to 2 do
11:   for  $j \leftarrow 0$  to 2 do
12:     if  $0 \leq (row\_i + i) < H$  and  $0 \leq (col\_i + j) < W$  then
13:        $\text{tileNs}[(ty * 2) + i][(tx * 2) + j] \leftarrow \text{img}[(row\_i + i) * W + col\_i + j]$ 
14:     else
15:        $\text{tileNs}[(ty * 2) + i][(tx * 2) + j] \leftarrow 0$ 
16:     end if
17:   end for
18: end for
19: Synchronize threads
20: if  $tx < \text{tile\_size}$  and  $ty < \text{tile\_size}$  then
21:    $\text{tmp}[2][2] \leftarrow 0.0f$ 
22:   for  $y \leftarrow 0$  to filter_size do
23:     for  $x \leftarrow 0$  to filter_size do
24:       for each p in tmp do
25:          $p \leftarrow p + \text{filter}[y][x] * \text{tileNs}[y + (ty * 2)][x + (tx * 2)]$ 
26:       end for
27:     end for
28:   end for
29:   for  $i \leftarrow 0$  to 2 do
30:     for  $j \leftarrow 0$  to 2 do
31:       if  $(row\_o + i) < H$  to  $(col\_o + j) < W$  then
32:          $\text{blur}[(row\_o + i) * W + (col\_o + j)] \leftarrow p\_value[i][j]$ 
33:       end if
34:     end for
35:   end for
36: end if
37: EndFunction

```

5 Experimental results

5.1 Experimental setup

The CPU implementation was conducted using OpenCV functions and then compared to custom CUDA implementations using CUDA Python (CuPy) raw kernels. CuPy is an open-source array library in Python that utilizes CUDA to perform GPU-accelerated computing. In addition to having pre-built functions, CuPy also allows the definition of three types of kernels: element-wise, reduction, and raw. These raw kernel models can be initialized, allowing developers to write and use a custom CUDA source code to enhance the performance of specific tasks [40].

The GPU kernel was then tested on two GPU hardware, 1660 Ti and Tesla T4. Both setups had the same computational capability but differed regarding available multiprocessors, cache sizes, and floating-point operations per second. More details on the hardware and software specifications are available in Table 3.

The algorithm was evaluated across various resolutions using images sourced from datasets such as [16–18]. These images were resized to resolutions ranging from 64×64 to $4k \times 4k$, simulating a wide range of practical scenarios. For each image, the lowest runtime was recorded out of multiple runtimes. The algorithm was then tested on additional image samples of the same size, and the average runtime across these samples was calculated for further analysis in all experiments conducted on both GPU and CPU.

5.2 Evaluation results

Initially, a comparison was made between the straightforward technique and CuDNN over the OpenCV CPU implementation for image sizes ranging from 64×64 to $4k \times 4k$ on two GPU devices, Nvidia GTX 1660 Ti and Nvidia Tesla T4. According to the results in Fig. 10, the speedup of CuDNN ranges from 1.8 to 4, while the speedup of the straightforward ranges from 1.8 to 9, which shows better performance of the straightforward over CuDNN in both GPU devices. However, the speedups for images larger than $1k \times 1k$ dropped more on the Tesla T4 GPU than the 1660 Ti. Despite the Tesla T4 offering faster floating-point operations and more multiprocessors, it showed less speedup than the 1660 Ti. This is because the Tesla T4 has a Error Correcting Code (ECC) by default which adds additional error-checking overhead at any memory access. Both figures showed a downward trend after $1k \times 1k$, potentially due to the limitations of cache sizes in handling simultaneous thread access to global memory. Despite having $1.5\times$ higher floating point TFLOPs and more cores than 1660 ti, the Tesla T4 gained less speedup due to ECC overhead. Therefore, optimization methods that rely less on caches and reduce memory accesses could help stabilize processing images across different hardware. The chosen optimization strategies aim to bring data closer to the processing units, reducing reliance on caches to handle excessive and redundant global memory reads. This prevents cache saturation, which could otherwise lead to a higher cache miss

Table 3 Hardware and software specifications

CPU setup	Intel Core i7-10750 H CPU @ 2.60GHz 32.0 GB RAM 12 MB Intel® smart cache 64-bit operating system, x64-based processor
GPU setup 1 (1660 Ti)	NVIDIA GeForce GTX 1660 Ti - 6GB Computing capability = 7.5 Multiprocessor count = 24 Threads per multiprocessor = 1024 Registers per multiprocessor = 64 KB L1 Cache per multiprocessor = 64 KB L2 Cache = 1.5 MB FP32 (float) = 5.437 TFLOPS
GPU setup 2 (Tesla T4)	NVIDIA Tesla T4 - 15GB Computing capability = 7.5 Multiprocessor count = 40 Threads per multiprocessor = 1024 Registers per multiprocessor = 64 KB L1 Cache per multiprocessor = 64 KB L2 Cache = 4 MB FP32 (float) = 8.141 TFLOPS
Python version	3.10.12
OpenCV version	4.8.0
CUDA version	11.8.0
CuPy version	13.0.0
Nvidia Nsight compute	2024.2.0.16

rate and less error checks that usually happen in the global memory accesses. For further demonstration, the straightforward method was chosen as the baseline.

The effectiveness of each optimization technique was demonstrated using the same images ranging from 64×64 to $4k \times 4k$ to evaluate the speedup of shared memory, R+MOPT, and SM + MOPT techniques over the straightforward implementation, as depicted in Fig. 11. For images larger than 128×128 , the SM + MOPT technique showed better performance, with speedups ranging from about 1.25 to 2. In contrast, R + MOPT speedup ranged from about 0.75 to 1.5, indicating that using shared memory to prefetch data and increasing the tasks for each thread improved performance. However, for images smaller than 128×128 , the SM technique showed better results, mainly because the number of threads did not surpass the threshold where MOPT techniques and their overhead would justify the effort to decrease the number of assigned blocks and block switches. Additionally, besides achieving results on the 5×5 filter suitable for this application, filters up to 9×9 were tested, and SM + MOPT continued to show better performance,

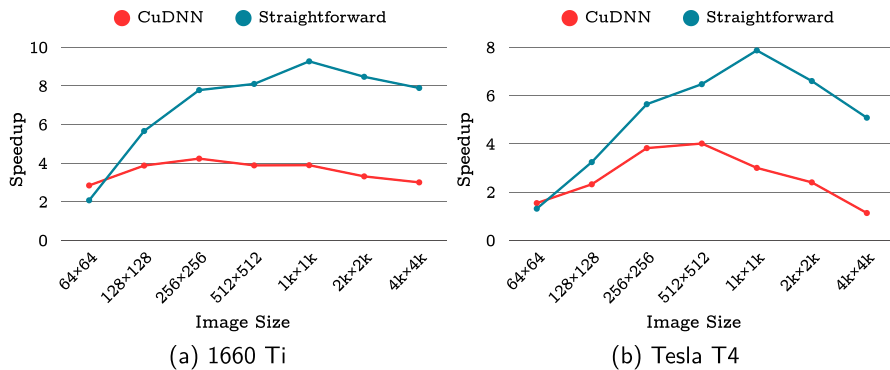


Fig. 10 Speedup of CuDNN and straightforward implementations over OpenCV CPU implementation for image sizes ranging from 64×64 to $4k \times 4k$ on two different GPU platforms

achieving up to about $2\times$ speedup over the straightforward implementation and up to $7.25\times$ over CuDNN. Concerning output quality, the mean squared error (MSE) between the CPU implementation and GPU implementations was near zero. Additionally, the applied optimizations did not impact image quality, as the outputs of the straightforward method and the optimized versions were identical.

To understand the effectiveness of each optimization technique, $4k \times 4k$ images were used as representatives for profiling with NVIDIA Nsight Compute [39], as highlighted in Table 4. The speedups were achieved due to a decrease in elapsed cycles and an increase in memory and DRAM throughput by shifting memory accesses to shared memory and registers instead of global memory. Additionally, by increasing the number of tasks per thread and computing more floating-point operations, the previously overutilized ALU (82.4%) was reduced, with the fused multiply-add pipeline becoming the highest-utilized at 36.1% based on active cycles. These changes led to a more balanced utilization between memory and computing units and decreased the total elapsed cycles of the kernel.

In another experiment, a $1k \times 1k$ input image was used to analyze the impact of varying filter sizes on the speedup achieved relative to the CPU implementation. The results showed a speedup of approximately $33\times$ at a filter size of 13×13 . As depicted in Fig. 12, the proposed *SM + MOPT* optimization demonstrated superiority over other techniques. However, as filter sizes increased, the efficiency declined due to the limited shared memory capacity. While the proposed method proves effective for the current application, instances requiring large convolution filters may encounter challenges with shared memory-oriented approaches.

The application-level speedup was measured while also implementing the masking and sharpening functions on the GPU. Parallelization for masking and sharpening was conducted using a straightforward implementation, where each output pixel was calculated within a thread, utilizing registers to hold subtraction and multiplication results. The final result was then stored in global memory. The measurement also accounted for the data transfer time between CPU and GPU, ensuring a

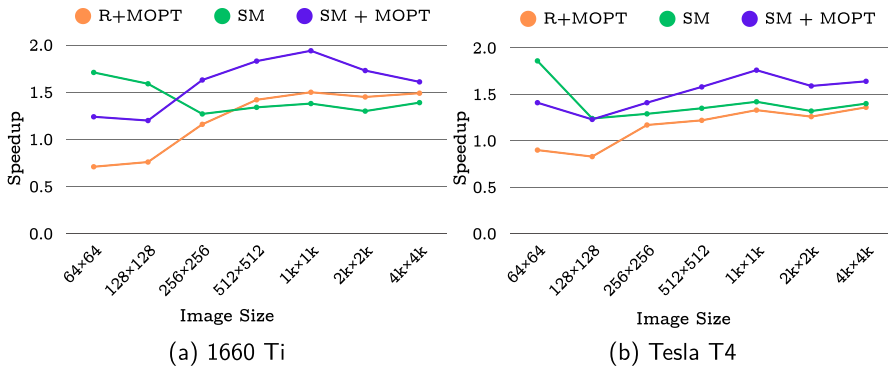


Fig. 11 Speedup of applying different optimization techniques over straightforward implementation for image sizes of 64×64 to $4k \times 4k$ on two different GPU platforms. **a** GPU hardware NVIDIA GTX 1660 Ti, **b** GPU hardware NVIDIA Tesla T4

Table 4 Profiling results of various convolution kernel implementations on an Nvidia Tesla T4 for a specific image size, $4k \times 4k$

Metric Name/GPU Kernel	Straightforward	SM	R+MOPT	SM+MOPT
Elapsed cycles	3,487,947	1,996,808	1,520,203	890,079
Memory throughput (%)	39.83	48.08	70.69	73.58
DRAM throughput (%)	2.79	8.61	17.02	15.97
L1/TEX cache throughput (%)	40.64	50.28	87.01	75.6
L2 cache throughput (%)	1.4	2.71	33.82	6.4
Compute throughput (%)	80.79	65.52	70.69	73.58

comprehensive evaluation of performance. The resulting speedups are reported in Table 5, showing an application speedup of up to $3.5\times$.

In another setup, the dash-cam 1080×1920 video presented in Fig. 2 is considered to be processed. Processing 5 min of this 30 FPS video initially required 4.68 min on the CPU, but GPU optimization significantly reduces this time, making it more suitable for real-time applications. The accelerated CPU-GPU application can process the video in only 1.54 min, enabling us to process it at a rate of about 97 FPS.

6 Conclusions

In the context of high-resolution image processing and the demand for real-time applications, certain computationally intensive algorithms require re-implementation to enhance performance and reduce latency. In this article, we selected a dark image local-global enhancement algorithm that exhibited a runtime bottleneck in the local enhancement process, specifically the convolution step, which accounted for 41.3% of the total runtime. We addressed this issue by leveraging

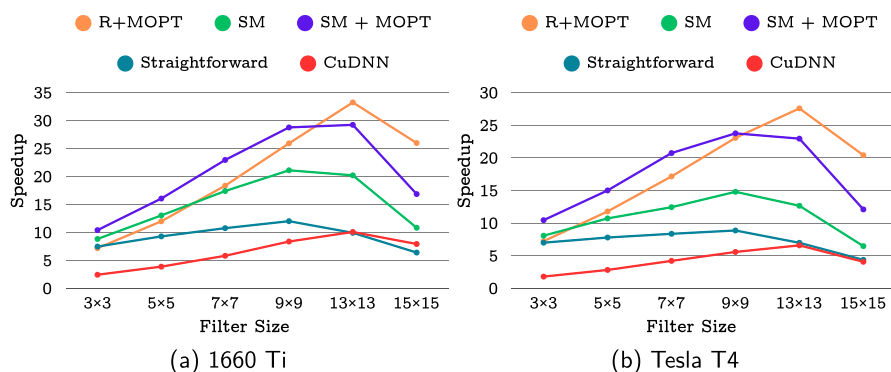


Fig. 12 Speedup achieved by applying various optimization techniques on GPU platforms, compared to the OpenCV CPU implementation for an image size of $1k \times 1k$ with filter sizes ranging from 3×3 to 15×15 . Two GPU platforms were used for evaluation: (a) NVIDIA GTX 1660 Ti and (b) NVIDIA Tesla T4

Table 5 Application-level speedups of the optimized local–global algorithm over the implementation on the CPU using OpenCV functions

Image name	City night [16]	License plate [19]	Street [17]	Satellite [18]
Image size (Pixels)	720×1080	1160×720	1024×2048	1768×5672
1660 Ti GPU	2.28×	2.09×	2.90×	3.26×
Tesla T4 GPU	3.24×	2.10×	2.91×	3.51×

GPU parallelization of the convolution function and employing optimization techniques that exploited constant memory, registers, and SM. This resulted in a $7.25\times$ speedup over CuDNN and a $16.36\times$ speedup over CPU OpenCV when tested on a 1660 Ti GPU. Finally, by merging masking and sharpening into a single GPU kernel along with the optimized convolution kernel, we achieved an application speedup of up to $3.5\times$ the CPU. Selecting the optimal combination of optimization techniques depends on the filter sizes commonly used in specific applications. For future work, an adaptive optimization approach could be developed to dynamically consider the application's requirements, such as filter size and available hardware resources, to determine the most efficient combination of optimizations for the given task.

Author contributions A.D implemented the algorithm on both CPU and GPU platforms. He also profiled this algorithm in order to understand which part is much more time-consuming in comparison with other parts. He wrote the draft of the paper. In addition, he prepared figures and tables. A.S proposed how the filter should be implemented in the GPU platform using CUDA parallel programming model. In addition, he modified the text, paper structure and organization and checked all concepts and results. He validated the obtained results and put them in a proper way in the paper. H.N collected the related data and papers. He summarized them and wrote some part of the paper. He provided some figures for the paper in order to show the importance of the mentioned algorithm. N.E helped in implementation part and collecting the materials, related papers and writing the paper. In addition, he prepared figures and tables. R.H checked

the obtained results and final submission. In addition, he helped for CPU implementation. All authors reviewed the paper.

Data availability No datasets were generated or analyzed during the current study.

Declarations

Conflict of interest The authors declare no conflict of interest.

References

1. Gonzalez RC, Woods RE (2018) Digital image processing. Pearson, London
2. Wang B, Xu H, Jiang G, Yu M, Ren T, Luo T, Zhu Z (2024) Uie-convformer: Underwater image enhancement based on convolution and feature fusion transformer. *IEEE Trans Emerg Top Computat Intell* 8(2):1952–1968
3. Amani N, Shahbahrami A, Nahvi M (2013) A new approach for face image enhancement and recognition. *Int J Adv Sci Technol* 52(01):1–10
4. Mandal G, Bhattacharya D, De P (2021) Real-time automotive night-vision system for drivers to inhibit headlight glare of the oncoming vehicles and enhance road visibility. *J Real-Time Image Process.* <https://doi.org/10.1007/s11554-021-01104-z>
5. Ghari B, Tourani A, Shahbahrami A, Gaydadjiev G (2024) Pedestrian detection in low-light conditions: a comprehensive survey. *Image Vis Comput* 148:105106. <https://doi.org/10.1016/j.imavis.2024.105106>
6. Turan B (2021) A image enhancement method for night-way images. *Balkan J Electr Comput Eng* 9(1):8–16. <https://doi.org/10.17694/bajece.802855>
7. Ngo D, Lee G-D, Kang B (2022) Singe image dehazing with unsharp masking and color gamut expansion. *IEEE Access* 10:102462–102474. <https://doi.org/10.1109/access.2022.3209665>
8. Mishra S, Vishwakarma A, Kumar A, Vimal R (2023) An automatic nuclei segmentation technique using unsharp masking. In: *IEEE International Conference on Computational Intelligence and Communication Networks*, pp 629–633. <https://doi.org/10.1109/cicn59264.2023.10402200>
9. Yang Y, Yun H (2022) An underwater image enhancement method based on color correction and local unsharp masking. In: *Journal of Physics: Conference Series*, vol 2400, p 012038. <https://doi.org/10.1088/1742-6596/2400/1/012038>
10. Zhang W, Wang Y, Li C (2022) Underwater image enhancement by attenuated color channel correction and detail preserved contrast enhancement. *IEEE J Oceanic Eng* 47(3):718–735. <https://doi.org/10.1109/joe.2022.3140563>
11. Kokro SK, Mwangi E, Kamucha G (2024) Histogram matching and fusion for effective low-light image enhancement. In: *SoutheastCon*, pp 200–206. <https://doi.org/10.1109/southeastcon52093.2024.10500161>
12. Yin TW, Shibghatullah ASB, Mansor NF et al (2022) Enhancement of low-light image using homomorphic filtering, unsharp masking, and gamma correction. *Int J Adv Comput Sci Appl.* <https://doi.org/10.14569/ijacsa.2022.0130864>
13. Carranza-Rojas J, Calderon-Ramirez S, Mora-Fallas A, Granados-Menani M, Torrents-Barrena J (2019) Unsharp masking layer: Injecting prior knowledge in convolutional networks for image classification. In: Tetko IV, Kůrková V, Karpov P, Theis F (eds) *Artificial Neural Networks and Machine Learning: Image Processing*, pp 3–16. https://doi.org/10.1007/978-3-030-30508-6_1
14. Gunnam S, et al (2023) Super resolution imaging based on residual network with generalized unsharp masking. In: *International Conference on Recent Trends in Advance Computing*, pp 557–561. <https://doi.org/10.1109/icrtac59277.2023.10480860>
15. Singh KB, Mahendra TV, Kurmvanshi RS, Rao CR (2017) Image enhancement with the application of local and global enhancement methods for dark images. In: *International Conference on Innovations in Electronics, Signal Processing and Communication*, pp 199–202. <https://doi.org/10.1109/iespc.2017.8071892>

16. Yang W, Yuan Y, Ren W, Liu J, Scheirer WJ, Wang Z, Zhang T et al (2020) Advancing image understanding in poor visibility environments: a collective benchmark study. *IEEE Trans Image Process* 29:5737–5752. <https://doi.org/10.1109/TIP.2020.2981922>
17. Nag S, Adak S, Das S (2019) What's there in the dark. In: *IEEE International Conference on Image Processing*, pp 2996–3000. <https://doi.org/10.1109/ICIP.2019.8803299>
18. Technologies M (2023) Nepal earthquake Nov. 2023 analysis-ready data. Maxar Open Data Program. Retrieved Jan 2024 from <https://www.maxar.com/open-data/nepal-earthquake-nov-2023>
19. Xu Z, Yang W, Meng A, Lu N, Huang H (2018) Towards end-to-end license plate detection and recognition: a large dataset and baseline. In: *Proceedings of the European Conference on Computer Vision*, pp 255–271. https://doi.org/10.1007/978-3-030-01261-8_16
20. Kirk DB, Wen-Mei WH (2023) Programming massively parallel processors: a hands-on approach. Elsevier, Cambridge. <https://doi.org/10.1016/c2020-0-02969-5>
21. Hijma P, Heldens S, Sclocco A, Van Werkhoven B, Bal HE (2023) Optimization techniques for GPU programming. *ACM Comput Surv* 55(11):1–81. <https://doi.org/10.1145/3570638>
22. Li H, Yang H, Zhao J (2017) An image-space parallel convolution filtering algorithm based on shadow map. In: *International Conference on Digital Image Processing*, vol 10420, pp 357–362. <https://doi.org/10.1117/12.2281728>
23. Jiang B (2018) Real-time multi-resolution edge detection with pattern analysis on graphics processing unit. *J Real-Time Image Process* 14(2):293–321. <https://doi.org/10.1007/s11554-014-0450-x>
24. Garea AS, Heras DB, Argüello F (2019) Caffe CNN-based classification of hyperspectral images on GPU. *J Supercomput* 75:1065–1077. <https://doi.org/10.1007/s11227-018-2300-2>
25. Akoushdeh A, Shahbahrani A, Joe Afshany A (2024) Parallelization of license plate localization on GPU platform. *Multimed Tools Appl* 83(1):2551–2564. <https://doi.org/10.1007/s11042-023-15656-8>
26. González-Ruiz V, Moreno JJ, Fernández JJ (2024) Acceleration of 3d feature-enhancing noise filtering in hybrid CPU/GPU systems. *J Supercomput* 80:12727–12742. <https://doi.org/10.1007/s11227-024-05928-x>
27. Chetlur S, Woolley C, Vandermersch P, Cohen J, Tran J, Catanzaro B, Shelhamer E (2014) cudnn: Efficient primitives for deep learning. *arXiv preprint arXiv:1410.0759*. <https://doi.org/10.48550/arXiv.1410.0759>
28. Hartung S, Shukla H, Miller JP, Pennypacker C (2012) Gpu acceleration of image convolution using spatially-varying kernel. In: *IEEE International Conference on Image Processing*, pp 1685–1688. <https://doi.org/10.1109/icip.2012.6467202>
29. Iandola FN, Sheffield D, Anderson MJ, Phothilimthana PM, Keutzer K (2013) Communication-minimizing 2d convolution in gpu registers. In: *IEEE International Conference on Image Processing*, pp 2116–2120. <https://doi.org/10.1109/ICIP.2013.6738436>
30. Van Werkhoven B, Maassen J, Bal HE, Seinstra FJ (2014) Optimizing convolution operations on GPUs using adaptive tiling. *Future Gener Comput Syst* 30:14–26. <https://doi.org/10.1016/j.future.2013.09.003>
31. Perrot G, Domas S, Couturier R (2016) An optimized GPU-based 2d convolution implementation. *Concurr Computat Pract Exp* 28(16):4291–4304. <https://doi.org/10.1002/cpe.3752>
32. Sanida T, Sideris A, Dasygenis M (2020) A heterogeneous implementation of the sobel edge detection filter using opencl. In: *International Conference on Modern Circuits and Systems Technologies*, pp 1–4. <https://doi.org/10.1109/mocast49295.2020.9200249>
33. Xiao H, Guo B, Zhang H, Li C (2021) A parallel algorithm of image mean filtering based on opencl. *IEEE Access* 9:65001–65016. <https://doi.org/10.1109/access.2021.3068772>
34. De Luca P, Galletti A, Marcellino L (2019) A gaussian recursive filter parallel implementation with overlapping. In: *International Conference on Signal-image Technology and Internet-based Systems*, pp 641–648. <https://doi.org/10.1109/sitis.2019.00105>
35. De Luca P, Galletti A, Marcellino L (2022) GPU-cuda implementation of the third order gaussian recursive filter. *SN Comput Sci* 3:1–11. <https://doi.org/10.1007/s42979-021-00960-7>
36. Lu G, Zhang W, Wang Z (2020) Optimizing GPU memory transactions for convolution operations. In: *IEEE International Conference on Cluster Computing*, pp 399–403. <https://doi.org/10.1109/cluster49012.2020.00050>

37. Lu G, Zhang W, Wang Z (2021) Optimizing depthwise separable convolution operations on GPUs. *IEEE Trans Parallel Distrib Syst* 33(1):70–87. <https://doi.org/10.1109/tpds.2021.3084813>
38. Guo Y, Lu L, Zhu S (2023) Novel accelerated methods for convolution neural network with matrix core. *J Supercomput* 79(17):19547–19573. <https://doi.org/10.1007/s11227-023-05399-6>
39. NVIDIA Corporation: NVIDIA Nsight Compute: Interactive Profiler for CUDA and NVIDIA OptiX. <https://developer.nvidia.com/nsight-compute>. Accessed: 2024-07-01
40. Preferred Networks, Inc: CuPy: GPU-Accelerated Computing with Python. <https://cupy.dev/>. Accessed: 2024-07-01

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Amin Daemdoost¹ · Asadollah Shahbahrami^{1,2} · Hesam Noorpoor¹ · Nima Esmi^{2,3} · Reza Hassanpour^{2,3}

✉ Asadollah Shahbahrami
shahbahrami@guilan.ac.ir

Amin Daemdoost
amindaemdoost@gmail.com

Hesam Noorpoor
hesamn2000@gmail.com

Nima Esmi
n.esmi.rudbardeh@rug.nl

Reza Hassanpour
r.zare.hassanpour@rug.nl

¹ Department of Computer Engineering, Faculty of Engineering, University of Guilan, Rasht, Iran

² Intelligent Systems Research Center, Khazar University, Baku, Azerbaijan

³ Bernoulli Institute for Mathematics, Computer Science and Artificial Intelligence, Groningen University, Groningen, The Netherlands